

Tareas

Tarea: Como obtener el ASCII de un letra en python

La función incorporada `ord()` permite obtener el número ordinal que está asociado con el carácter especificado. Notar que <Código ASCII> se emplea en un sentido abarcativo, pues incluye, en efecto, caracteres unicode.

```
>>> ord("a")  
97
```

Para recuperar un carácter a partir de su ordinal, se emplea `chr()`.

```
>>> chr(97)  
'a'
```

Investigación PCA con T-SNE

TAREA: Investigar a fondo los pasos para aplicar PCA. Además investigar t-SNE, decir sus diferencias y dónde se aplica cada uno.

1) PCA (ANÁLISIS DE COMPONENTES PRINCIPALES)

Es una técnica LINEAL que transforma las variables originales en nuevas variables llamadas componentes principales (CP), que NO están correlacionadas entre sí y que capturan la mayor varianza posible. La CP1 tiene la mayor varianza, la CP2 la segunda, etc.

PASOS

Paso 1: Se arma la matriz de datos X de dimensión $m \times n$ (m = muestras, n = variables)

Ejemplo: 5 muestras, 2 variables

$$X = \begin{bmatrix} [2,1], & [3,3], & [4,2], & [5,5], & [6,4] \end{bmatrix}$$

Paso 2: Se centran / estandarizan los datos (cada variable con media 0 y varianza 1)

$$z = (x - \mu) / \sigma$$

(si las variables están en distinta escala es OBLIGATORIO estandarizar, si no la variable más grande domina)

$$\mu = [4, 3] \rightarrow \bar{z} = X - \mu = \begin{bmatrix} [-2, -2], & [-1, 0], & [0, -1], & [1, 2], & [2, 1] \end{bmatrix}$$

Paso 3: Se calcula la matriz de covarianza c ($n \times n$)

$$c = (1 / (m-1)) \cdot \bar{z}^T \bar{z} \rightarrow c = \begin{bmatrix} [2.5, 2], & [2, 2.5] \end{bmatrix}$$

Paso 4: Se obtienen eigenvalores (λ) y eigenvectores (v) de c

$$c \cdot v = \lambda \cdot v \rightarrow \det(c - \lambda I) = 0$$

$$(2.5 - \lambda)^2 - 4 = 0 \rightarrow \lambda_1 = 4.5, \lambda_2 = 0.5$$

$$v_1 = [0.707, 0.707] \quad v_2 = [-0.707, 0.707]$$

los eigenvectores son las DIRECCIONES de las cp y los eigenvalores la VARIANZA en esa dirección

paso 5: Se ordenan los eigenvalores de mayor a menor y se calcula la varianza explicada

$$\text{var. explicada}_i = \lambda_i / \sum \lambda \rightarrow \text{CP1} = 4.5/5 = 90\% , \text{CP2} = 0.5/5 = 10\%$$

paso 6: Se eligen K componentes ($K < n$)

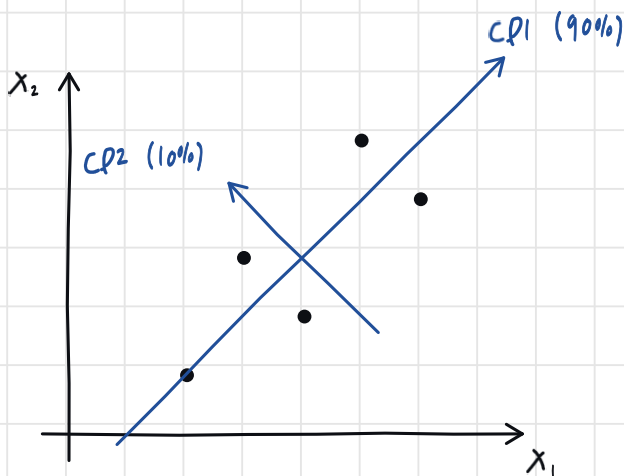
criterio: varianza acumulada $\geq 90-95\%$ ó "método del codo" en el scree plot

aquí $K = 1$ porque la CP1 ya explica el 90%

paso 7: Se proyectan los datos sobre las K componentes

$$y = \bar{z} \cdot W_K \quad (W_K = \text{matriz con los K eigenvectores, dimensión } n \times K)$$

$$y = [-2.83, -0.71, -0.71, 2.12, 2.12] \rightarrow \text{pasamos de 2D a 1D}$$



la CP1 va en la dirección donde los datos se "estiran" más (máxima varianza)

En código:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
x_std = StandardScaler().fit_transform(X)
pca = PCA(n_components=0.95) # o n_components = K
y = pca.fit_transform(x_std)
pca.explained_variance_ratio_
```

Rápido, determinista e interpretable | pero solo capta relaciones LINEALES

Ventaja

Desventaja

2) t-SNE (t-DISTRIBUTED STOCHASTIC NEIGHBOR EMBEDDING)

Es una técnica NO LINEAL y probabilística (van der Maaten y Hinton, 2008) pensada principalmente para VISUALIZAR datos de alta dimensión en 2D o 3D. Lo que busca es que los puntos que eran vecinos en el espacio original sigan siendo vecinos en el mapa (conserva estructura LOCAL).

PASOS

Paso 1: Se calculan similitudes en alta dimensión con una gaussiana centrada en cada punto

$$p_{ji} = \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2) / \sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)$$

o se ajusta con la PERPLEJIDAD ($\approx$ número efectivo de vecinos, típico 5 a 50)

Paso 2: Se simetrizan las probabilidades

$$p_{ij} = (p_{ji} + p_{ij}) / 2m$$

Paso 3: Se inicializan los puntos y_i en 2D (aleatorio o con PCA)

Paso 4: Se calculan similitudes en baja dimensión con una t-Student de 1 grado de libertad

$$q_{ij} = (1 + \|y_i - y_j\|^2)^{-1} / \sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}$$

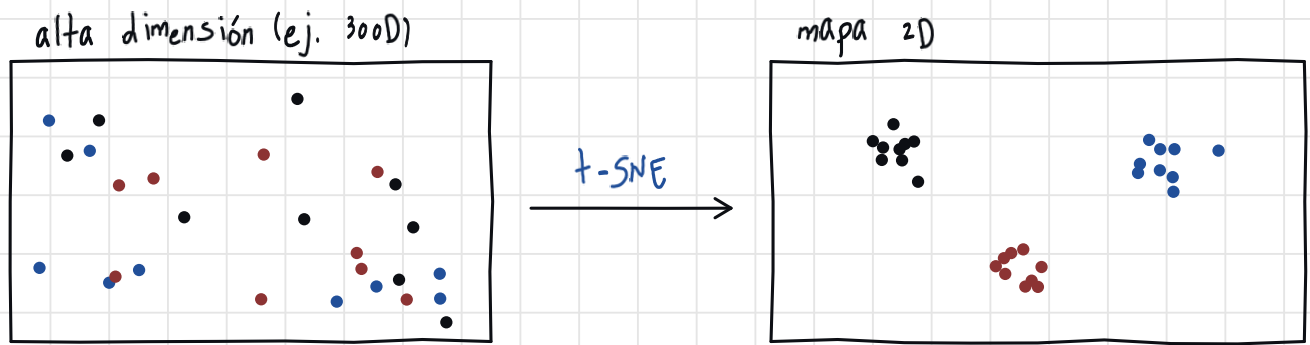
la t-Student tiene "colas pesadas" $\rightarrow$ separa mejor los clusters y evita el "crowding problem"

Paso 5: Se minimiza la divergencia de Kullback-Leibler entre p y q

$$KL(p||q) = \sum_i \sum_j p_{ij} \cdot \log(p_{ij} / q_{ij})$$

Paso 6: Se actualizan los y_i con descenso de gradiente hasta converger (≈ 1000 iteraciones)

$$\partial KL / \partial y_i = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$



En código:

```
from sklearn.manifold import TSNE  
y = TSNE(n_components=2, perplexity=30,  
init="pca", random_state=42).fit_transform(X)
```

Muestra muy bien los clusters | pero es lento y aleatorio

^{Ventaja} OJO: en t-SNE el tamaño de los clusters ^{Desventaja} y la distancia ENTRE clusters no significan nada, solo importan los vecinos

3) DIFERENCIAS PCA vs t-SNE

	PCA	t-SNE
Tipo	Lineal	No lineal
Qué conserva	varianza / estructura GLOBAL	vecindades / estructura LOCAL
Método	Álgebra lineal (eigen / SVD)	optimización (min KL con gradiente)
Resultado	Determinista (siempre igual)	Estocástico (cambia con la semilla)
Datos nuevos	Sí, con <code>.transform()</code>	No, hay que recalcular todo
Costo	Rápido, escala a muchos datos	Lento, $\approx O(m^2)$ (Barnes-Hut $m \cdot \log m$)
Dimensión salida	cualquier k	Normalmente 2 o 3
Interpretación	Sí (cargas / varianza explicada)	Solo visual
Hiperparámetros	Solo k	perplejidad, learning rate, iter.
Uso principal	preprocesamiento / compresión	visualización / exploración

4) ¿DÓNDE SE APLICA CADA UNO?

PCA:

- Antes de entrenar un modelo para reducir variables y quitar multicolinealidad
- Comprimir imágenes y reconocimiento facial (eigenfaces),
- Quitar ruido de los datos (se quedan las CP con más varianza)
- En PLN: reducir vectores enormes como one-hot / TF-IDF (LSA usa SVD, que es la misma idea)
- Finanzas: encontrar factores de riesgo en carteras y tasas de interés

t-SNE:

- Visualizar word embeddings (word2vec, GloVe) y ver qué palabras quedan juntas
- Ver clusters de documentos o tópicos en PLN
- Explorar datos antes de hacer clustering (ej. MNIST, dígitos escritos a mano)
- Biología / genómica: visualizar tipos de células (single-cell RNA-seq)

conclusión:

PCA sirve para REDUCIR dimensiones y usar el resultado en otro modelo; t-SNE sirve para VER los datos. En la práctica se combinan: primero PCA para bajar a ≈ 50 dimensiones (quita ruido y acelera) y luego t-SNE para bajar a 2D y graficar.